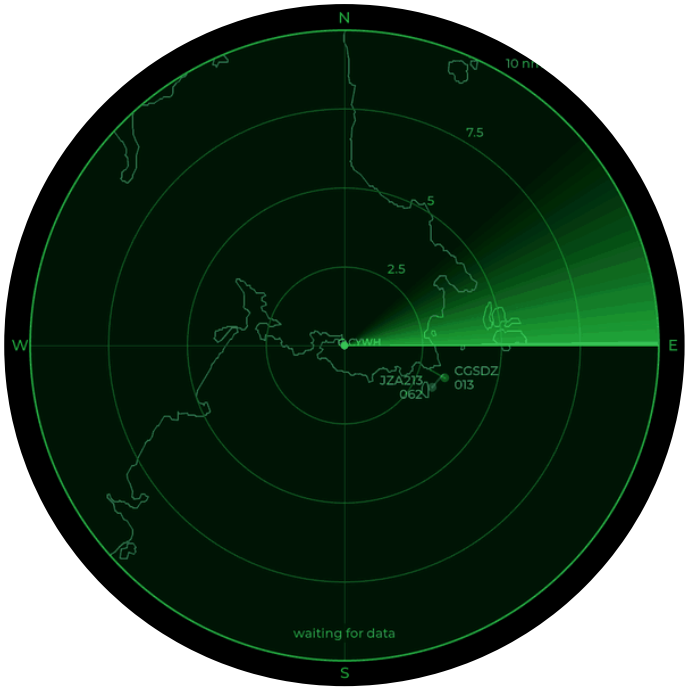
A radar for your own sky

LVGL on a Raspberry Pi framebuffer: from "touch the screen" to a live ADS-B radar display, with no X or Wayland.

Gordon M. Celesta · Victoria, BC · Raspberry Pi meetup, October 2026



This is the write-up for my talk. It's meant to be followed along with during the Meetup, and to be enough to build your own afterwards. The code is at github.com/va7sdf/lvgl-radar.

There are two projects in it:

- **The touch demo**, about 200 lines of C. It puts a picture on a screen and reads touches with no desktop at all. It's the "hello, world" for any screen on a Pi, round, square or a little SPI panel.
- **The radar**, which plots the aircraft my receiver hears as if on an old radar scope. The sweep turns with a phosphor glow, aircraft move only as it passes, and the coastline of southern Vancouver Island glows underneath. Touch an aircraft and you get its details; swipe for weather, receiver stats, a list and settings.

Contents

1. Why skip the desktop? The framebuffer, evdev and LVGL

2. The hardware Parts, setting up the screen and finding the touch panel
3. The touch demo, step by step Build it and walk through the code
4. Building the radar From a fresh SD card to a running radar
5. How the radar works Data flow, threads, the sweep and the tracks
6. The map Shorelines, rivers, airports and navaids for your location
7. Touch and screens Every gesture and screen, and two touch bugs worth knowing
8. Fonts and icons TrueType at runtime, a seven-segment clock and weather icons
9. Where the data comes from The receiver and five free web services
10. Settings radar.conf, units and alerts
11. Running it for real A service, taking it out, and showing it on a projector
12. Credits, lessons and ideas
13. Glossary The aviation and computing terms used here

In one minute

Computer	Raspberry Pi Zero 2 W, Raspberry Pi OS Lite (64-bit), no desktop
Screen	Waveshare 5" round, 1080 × 1080, HDMI, 10-point USB touch
Graphics	LVGL 9.3 drawing straight to `/dev/fb0`
Touch	Linux evdev, through LVGL's driver
Language	C (the touch demo); C and a little Python (the radar's tools)
Aircraft	My own dump1090-fa receiver, plus the adsb.fi open data feed
Network	IPv4 or IPv6 (chapter 9 has the details)
Licence	BSD 2-Clause; fonts and data keep their own licences

1. Why skip the desktop?

A Raspberry Pi with a desktop runs a display server (X11 or Wayland), a window manager, a panel, and then your program in a window. That's the right choice for a general-purpose computer. A lightweight window manager such as Openbox trims away the panel and the extras, but the display server is still there underneath. For a gadget with one job, like a clock, a dashboard or a radar, it's a lot of machinery between your program and the glass:

- **Memory.** The Pi Zero 2 W has 512 MB. A desktop takes a good share of that before your program starts.
- **Boot time.** Pi OS Lite goes straight from boot to your app, with no desktop to load first.
- **Simplicity.** No auto-login, no hiding the mouse pointer, no screen blanking to fight with, no window to make full-screen.

Linux already has everything needed to draw and to read touches without any of it.

The framebuffer: the screen as a file

Everything in Unix-like systems is a file, and the screen is no exception: `/dev/fb0` is the screen's memory, presented as a file. Write bytes to it and pixels change. On this screen each pixel is two bytes in a format called RGB565 (5 bits red, 6 green, 5 blue), so 1080 × 1080 pixels is 2,332,800 bytes. You can try it from the shell:

```
cat /sys/class/graphics/fb0/virtual_size     # 1080,1080
cat /sys/class/graphics/fb0/bits_per_pixel   # 16
cat /dev/urandom > /dev/fb0                  # snow! (Ctrl-C)
```

It even works from Python:

```
# Paint the top 100 rows green: RGB565 green is 0x07E0
with open("/dev/fb0", "wb") as fb:
    fb.write(b"\xe0\x07" * 1080 * 100)
```

Drawing text, buttons and animations by poking bytes gets old quickly, though. That's what LVGL is for.

evdev: touches as a stream of events

Input devices are files too. Every one (keyboard, mouse, touch screen) appears under `/dev/input/` as an *event device*. Reading it gives a stream of small records: "X is now 2048", "Y is now 1310", "finger down", "sync". The `evtest` tool prints them, which is the best way to check a touch screen works (chapter 2).

LVGL: a GUI library for small screens

LVGL (Light and Versatile Graphics Library) is a C library for graphical interfaces on microcontrollers and small Linux boards. It's MIT-licensed and used in everything from smart watches to 3D printers. It gives you:

- **Widgets**: labels, buttons, sliders, lists, charts, tabs, canvases and more
- **Styles**: colours, borders, radius, shadows, opacity, like a small CSS
- **Events**: pressed, clicked, long-pressed, gestures (swipes), value changed
- **Fonts**: built-in, converted bitmaps, or TrueType rendered at runtime
- **Drivers**: for the Linux framebuffer and evdev, among many others

LVGL never talks to the hardware directly. It draws into a buffer in memory and hands the changed rectangles to a *display driver*, which here copies them into `/dev/fb0`. Touches come from an *input driver*, here evdev. Swap the drivers and the same program runs on an ESP32, in an SDL window on your laptop, or on a Linux DRM display.

The main loop

An LVGL program builds its screens, then loops forever, letting LVGL do whatever is due:

```
for (;;) {
    uint32_t idle = lv_timer_handler();   // redraw, read touch, run timers
    usleep(idle * 1000);                  // sleep until the next job
}
```

Python folks will recognise it: it's an event loop, like `asyncio` or Tkinter's `mainloop()`. Your code runs in *callbacks*: functions LVGL calls when a button is pressed or a timer fires.

Configuring LVGL

LVGL is compiled with your program, and a header file, `lv_conf.h`, switches its features on and off. The ones that matter here:

```
#define LV_COLOR_DEPTH      16   // match the framebuffer's RGB565
#define LV_USE_LINUX_FBDEV   1   // display driver: /dev/fb0
#define LV_USE_EVDEV         1   // input driver: /dev/input/event*
#define LV_USE_TINY_TTF      1   // TrueType fonts at runtime (chapter 8)
#define LV_FONT_MONTSERRAT_24 1  // built-in fonts, one per size
```

The repo has one `lv_conf.h` at the top, shared by both projects.

The console gets in the way

The Linux text console also draws on the framebuffer. Start a graphical program from it and the console's blinking cursor keeps punching a hole in your picture. The fix is one `ioctl` call that tells the console to stand back, and another to give the screen back on the way out:

```
int tty = open("/dev/tty1", O_RDWR);
ioctl(tty, KDSETMODE, KD_GRAPHICS);   // console: hands off the screen
/* ... run ... */
ioctl(tty, KDSETMODE, KD_TEXT);       // console back (also on Ctrl-C)
```

It needs root, or the `CAP_SYS_TTY_CONFIG` capability, which is how the radar's systemd service does it without running as root (chapter 11).

What about Python?

LVGL has an official MicroPython binding, and on a full Pi there are other routes to a desktop-free screen: pygame (SDL2) can draw through KMS/DRM, and simple dashboards can write images straight to `/dev/fb0` with Pillow. I chose C and LVGL for speed on the Zero 2 (the radar sweep redraws a lot of pixels many times a second) and because the same skills carry over to microcontrollers.

2. The hardware

Parts

Part	Notes
Raspberry Pi Zero 2 WH	The "H" has the GPIO header soldered on. Any Pi works; a Pi 4 or 5 has power to spare
Waveshare 5" round LCD, 1080 × 1080	HDMI for the picture, USB for the 10-point capacitive touch
Mini-HDMI to HDMI cable (or adapter)	The Zero has a mini-HDMI socket
Micro-USB OTG cable	From the Zero's USB data port to the screen's touch port
5 V 2.5 A power supply	Into the Zero's power port
A second 5 V supply	For the screen, into its power port
microSD card, 16 GB or more	Raspberry Pi OS Lite (64-bit)
An ADS-B receiver (optional)	Mine is dump1090-fa on another Pi. Without one, the radar can use adsb.fi alone

I power the screen from a second Raspberry Pi power supply I had to hand. The screen can also take power through its touch USB port. Whether one supply can run both the Pi and the screen that way, with touch still working, is something to try for yourself.

> **Lesson learned:** when the screen went blank one evening, after a lot of head-scratching the mini-HDMI plug turned out to be half out of the socket. Check the cable first.

Install Raspberry Pi OS Lite

Use Raspberry Pi Imager to write **Raspberry Pi OS Lite (64-bit)**. In its settings, set the hostname (mine is `radar`), your user, Wi-Fi and SSH, so the Pi is on the network on first boot and you can work over SSH.

Tell the Pi about the round screen

A square 1080 × 1080 HDMI panel isn't a mode the Pi guesses on its own. Add this to the end of the single line in `/boot/firmware/cmdline.txt` (not on a new line):

```
video=HDMI-A-1:1080x1080@60
```

> **Lesson learned:** guides for older Pi OS versions put `hdmi_group`, `hdmi_mode` and `hdmi_timings` in `config.txt`. With today's KMS graphics driver the `video=` setting in `cmdline.txt` is what counts.

Keep boot messages off the screen

By default the kernel prints its boot messages on the screen, and they can end up showing around the edges of the app. In the same line of `cmdline.txt`, change `console=tty1` to `console=tty3`, so the messages go to a console nobody looks at, and add:

```
quiet loglevel=3 logo.nologo vt.global_cursor_default=0
```

These print only serious kernel messages, drop the Raspberry Pi logos at boot, and hide the console's blinking cursor. Leave `console=serial0,115200` alone if it's there, and keep everything on one line. You can still see the boot messages with `dmesg` or `journalctl -b`.

Reboot, and check the framebuffer agrees:

```
cat /sys/class/graphics/fb0/virtual_size     # should say 1080,1080
cat /sys/class/graphics/fb0/bits_per_pixel   # 16
```

The screen's backlight button cycles brightness, and one press turns it off. If the picture vanishes, it may just be dark.

Find the touch panel

Install `evtest`, run it, and pick the Waveshare device from its list:

```
sudo apt install evtest
sudo evtest
```

Touch the screen and you'll see `ABS_X`, `ABS_Y` and `BTN_TOUCH` events scroll by. Corners show the range of the numbers: this panel reports roughly 0 to 4095 in each direction, which LVGL scales to 0–1079.

Device numbers like `event2` can change between boots, so it's better to use the stable name under `/dev/input/by-id/`:

```
ls /dev/input/by-id/
# usb-Waveshare_Waveshare_-079-HD-event-if00
```

Permissions

Your user needs to be in the `video` group (the framebuffer) and the `input` group (the touch screen). The default Pi OS user is in both. Check with `groups`.

3. The touch demo, step by step



Touch the screen and it shows where: crosshairs, a ring under your finger, the X and Y coordinates in big type, a count of touches, and a trail of dots as you drag. It's the first thing to get working with any new screen, because it proves the whole chain: the display mode, the framebuffer, the touch panel, its scaling, and LVGL.

Build and run

```
sudo apt install git cmake build-essential
git clone --recurse-submodules https://github.com/va7sdf/lvgl-radar.git
cd lvgl-radar/touch-demo
cmake -B build
cmake --build build -j4
sudo ./build/touch-demo
```

The first build compiles LVGL itself, which takes a few minutes on a Zero 2; after that, rebuilds take seconds. **Ctrl-C** quits and gives the text console back.

`--recurse-submodules` matters: LVGL is a *git submodule*, a pointer to a specific version (9.3.0) of LVGL's own repository. If you cloned without it, run `git submodule update --init`.

> **IPv6 only?** github.com has no IPv6 address yet, so cloning from a network with IPv6 and no IPv4 needs NAT64/DNS64 (chapter 9). Most IPv6-only networks, including mobile carriers, provide it.

The demo finds the touch screen itself. To choose a device, name it:

```
sudo ./build/touch-demo /dev/input/by-id/usb-Waveshare_Waveshare_-079-HD-event-if00
```

`sudo` is only there to hide the console cursor (chapter 1). Without it the demo still runs, with a blinking cursor in the corner.

Walking through the code

Everything is in `touch-demo/main.c`. Here are the parts that matter.

Start LVGL and connect the screen

```
lv_init();
lv_tick_set_cb(tick_ms);                    // how LVGL tells the time

lv_display_t *disp = lv_linux_fbdev_create();
lv_linux_fbdev_set_file(disp, "/dev/fb0");  // draw here
```

LVGL asks the framebuffer for its size and pixel format, so the same code works on any screen.

Connect the touch screen

```
if (argc > 1)
    lv_evdev_create(LV_INDEV_TYPE_POINTER, argv[1]);   // the one you named
else
    lv_evdev_discovery_start(found_device, NULL);      // find them all
```

Discovery watches `/dev/input` and adds every touch screen, mouse and keyboard it finds, including ones plugged in later. LVGL reads each touch panel's range from the kernel and scales it to the screen's pixels.

Build the screen

Widgets are created as children of a parent, here the screen itself:

```
coords = lv_label_create(scr);
lv_obj_set_style_text_font(coords, &lv_font_montserrat_48, 0);
lv_obj_set_style_text_color(coords, lv_color_hex(0xC8FFD0), 0);
lv_label_set_text(coords, "Touch the screen");
lv_obj_align(coords, LV_ALIGN_CENTER, 0, -20);
```

The crosshairs are two `lv_line` widgets, the ring is a plain object with a round border, and the trail is 120 small round objects created up front, hidden, and reused oldest-first. Creating and deleting widgets on every touch would work too, but reusing them keeps memory steady.

React to touches

One function handles every touch event on the screen:

```
lv_obj_add_event_cb(scr, on_touch, LV_EVENT_PRESSED, NULL);
lv_obj_add_event_cb(scr, on_touch, LV_EVENT_PRESSING, NULL);
lv_obj_add_event_cb(scr, on_touch, LV_EVENT_RELEASED, NULL);
```

`PRESSED` is the finger going down, `PRESSING` repeats while it stays down (and moves), and `RELEASED` is it lifting. Inside, ask where the finger is and move things there (slightly simplified):

```
static void on_touch(lv_event_t *e)
{
    lv_point_t p;
    lv_indev_get_point(lv_indev_active(), &p);   // where is the finger?

    if (lv_event_get_code(e) == LV_EVENT_RELEASED) {
        lv_obj_add_flag(ring, LV_OBJ_FLAG_HIDDEN);   // finger up: hide
        return;
    }
    lv_obj_set_pos(ring, p.x - 40, p.y - 40);        // centre the ring
    lv_obj_remove_flag(ring, LV_OBJ_FLAG_HIDDEN);
    lv_label_set_text_fmt(coords, "X %d   Y %d", (int)p.x, (int)p.y);
}
```

Note that nothing here *draws*. Callbacks only change widgets; LVGL works out which parts of the screen changed and redraws just those, the next time round the main loop.

The main loop

```
for (;;) {
    uint32_t idle = lv_timer_handler();
    usleep((idle > 33 ? 33 : idle) * 1000);
}
```

Capping the sleep at 33 ms means the touch screen is read about 30 times a second, even when nothing else is due.

Things to try

- Change the colours (`lv_color_hex(0x...)`) or the font size (`lv_font_montserrat_24` or `_48`; the sizes compiled in are set in `lv_conf.h`).
- Add a button: `lv_button_create(scr)` with a label child, and an `LV_EVENT_CLICKED` callback that clears the trail.
- Run it on another screen: an SPI panel that appears as `/dev/fb1` works with `FBDEV=/dev/fb1 sudo ./build/touch-demo`.

4. Building the radar

With the touch demo working, the radar needs only a few more packages and some settings.

Build it

```
sudo apt install git cmake build-essential libcurl4-openssl-dev libcjson-dev
git clone --recurse-submodules https://github.com/va7sdf/lvgl-radar.git
cd lvgl-radar/radar
cmake -B build
cmake --build build -j4
```

`libcurl` fetches data from the web and `cJSON` reads it. Both are standard Pi OS packages.

Set your position and receiver

Open `radar.conf` in your editor (it's commented throughout). The first things to set:

```
lat = 48.4220           # your receiver's position, decimal degrees
lon = -123.3700         # (this is Victoria's Inner Harbour: use your own)
range_nm = 10           # starting range: outer ring, nautical miles

[source]
name = local
url = http://192.168.1.10/data/aircraft.json    # your receiver
array_key = aircraft
poll_s = 1
colour = 48FF6A
```

Where `aircraft.json` lives depends on the receiver software. Common places, with `PI` your receiver's address:

Software	URL
dump1090-fa with SkyAware	`http://PI/skyaware/data/aircraft.json`
readsb / tar1090	`http://PI/tar1090/data/aircraft.json`

Open the URL in a browser: a page of JSON listing aircraft means it's right.

The address can be IPv4 or IPv6. An IPv6 address goes in square brackets, `url = http://[fd00::22]/data/aircraft.json`, or use a hostname: if it has both kinds of address, the radar tries IPv6 first and falls back to IPv4. The receiver's web server has to be listening on IPv6 for that to work. No receiver? Delete the `local` source and keep `adsb.fi` (chapter 9).

Make the map

The outline map is made for your position by a Python script that downloads free shoreline and airport data the first time:

```
./make_map.py --download
```

It writes `map.txt`, about 300 KB for 50 nm around Victoria; chapter 6 has the options. You can also run it later from the radar's Settings screen with **Update map**.

Run it

```
sudo ./build/radar radar.conf
```

The sweep starts at once; aircraft appear as their data arrives and the sweep passes them. Ctrl-C stops it. When you're happy, make it start at every boot:

```
./install-service.sh
```

That installs and starts a systemd service (chapter 11 explains it). From then on, after changing code or settings:

```
cmake --build build -j4 && sudo systemctl restart radar
journalctl -u radar -f          # its output, live
```

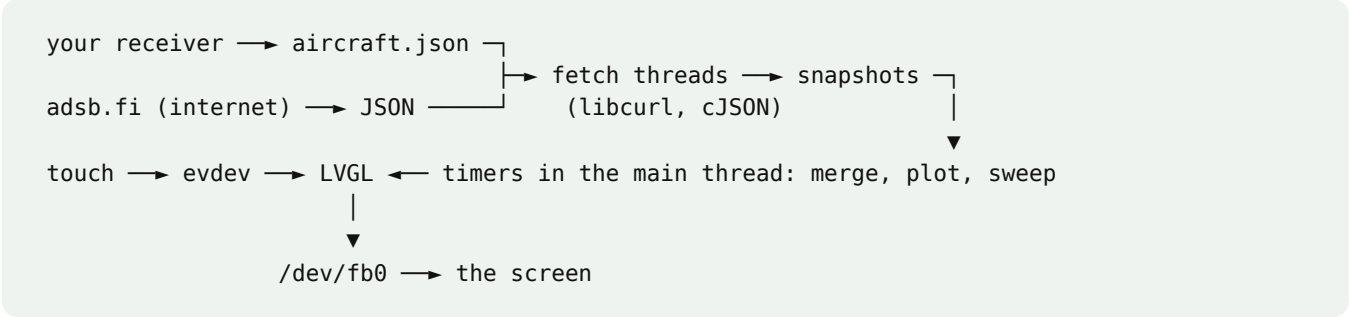
Updating

```
git pull
git submodule update            # in case LVGL moved
cmake --build build -j4 && sudo systemctl restart radar
```

Your edits to `radar.conf` survive a `git pull` unless the same lines changed upstream; git will tell you if so.

5. How the radar works

The big picture



ADS-B (Automatic Dependent Surveillance–Broadcast) is how aircraft announce themselves: about twice a second each one broadcasts its identity, position, altitude, speed and heading on 1090 MHz. A receiver such as dump1090 decodes those messages from a cheap USB radio and publishes the current picture as `aircraft.json`, a file that updates every second.

The radar reads that file, and adsb.fi's equivalent for the same area, and draws what's in it.

Threads: don't keep the screen waiting

A web request can take a moment, or hang for seconds when Wi-Fi drops. If the screen's loop waited for it, the sweep would stutter and touches would be ignored. So each data source has its own **thread**: a loop that fetches, parses, stores the result and sleeps until the next poll.

The one firm rule: **LVGL may only be used from the main thread.** It's not built for several threads changing widgets at once. So fetch threads never touch the screen. They put each new list of aircraft in a *snapshot*, guarded by a mutex (a lock), and the main thread copies the latest snapshot every half second from an LVGL timer:

```
lv_timer_create(refresh_cb, 500, NULL);   // every 500 ms, in the main thread

static void refresh_cb(lv_timer_t *t)
{
    for (int s = 0; s < cfg.n_sources; s++)
        feed_get(s, &snaps[s], ...);      // copy under the lock
    merge_sources();                      // one list, first source wins
    radar_ui_update(&merged);             // move the aircraft
    ui_update(&merged);                   // the other screens
}
```

In Python terms: worker threads put results in a `queue`, and the GUI thread takes them out. Weather, METARs (Meteorological Aerodrome Reports, the standard airport weather reports), receiver stats and aircraft lookups work the same way, each in its own thread.

Merging two sources

My receiver and adsb.fi often see the same aircraft. They're matched by their **ICAO address**, a 24-bit number unique to each airframe, shown as six hex digits like `C0184B`. (ICAO, the International Civil Aviation Organization, is the United Nations agency that sets the world's aviation standards, from these addresses to airport codes like CYYJ.) The first source in `radar.conf` wins (put your own first), and it borrows details only adsb.fi has, such as registration, type, owner and the military flag. Each aircraft is drawn in its source's colour, so you can see what your own antenna is hearing.

From latitude and longitude to pixels

Over 25 nautical miles the Earth is flat enough. One minute of latitude is one nautical mile; a minute of longitude is a little shorter, by the cosine of the latitude:

```
double east_nm  = (lon - home_lon) * 60.0 * cos(home_lat * M_PI / 180.0);
double north_nm = (lat - home_lat) * 60.0;
int x = 540 + east_nm  * 500 / range_nm;    // 500 px = outer ring
int y = 540 - north_nm * 500 / range_nm;    // screen y grows downwards
```

The sweep

The sweep is the most demanding part: a wedge of fading green, hundreds of pixels long, turning smoothly 20 times a second on a Zero 2.

Drawing it with LVGL's shapes would mean redrawing the whole disc every frame, which is too slow. Instead the face (disc, map, rings, labels) is drawn once into a **canvas**, a widget that's just a block of pixels the program can write to directly. Two copies are kept: the face as it should look without the sweep, and what's on screen.

At start-up a **lookup table** is built holding, for every pixel, its bearing from the centre. Each frame, only the pixels the sweep has moved across are recomputed, and for each one the work is tiny:

```
uint16_t d = angle - bearing[pixel];      // how far behind the sweep edge
if (d < trail)
    screen[pixel] = blend(face[pixel], green, fade[d]);   // glowing
else
    screen[pixel] = face[pixel];          // back to the plain face
```

No trigonometry per frame, just a subtraction, a table lookup and a colour blend. Angles are 16-bit numbers where 65,536 is a full circle, so they wrap around north by themselves. The Settings screen shows how long each sweep frame takes to draw, against the time available per frame.

Aircraft move only when swept

A real radar only learns where an aircraft is when the beam passes it, so the blip jumps once per turn and glows, then fades until the next pass. The radar copies that. New positions are stored when they arrive, but the blip moves only when the sweep crosses its bearing:

1. The sweep crosses the aircraft: move the blip to the latest position, set its glow to full, add a point to its track.
2. Every frame after: fade it a little (the *phosphor*).
3. If the aircraft has gone from the data when the sweep comes round, remove it and its track.

It's close to a real radar, not exact. Each aircraft reports its position about twice a second; the receiver publishes it once a second, adsb.fi is asked every few

seconds, and the screen catches up every half second. So a blip shows where the aircraft was a second or few ago, not at the instant the sweep passes. At these ranges that's a few hundred metres at most for an airliner: close enough to watch the traffic, not for keeping aircraft apart.

Tracks

Each aircraft keeps a history of where it's been (two minutes by default) and draws it as a line that fades with age. Tracks are drawn into the canvas under the sweep, so the sweep glides over them. When an aircraft leaves the scope, drops out of range of the receivers, or turns off its transponder, its track goes with it.

Colours that mean something

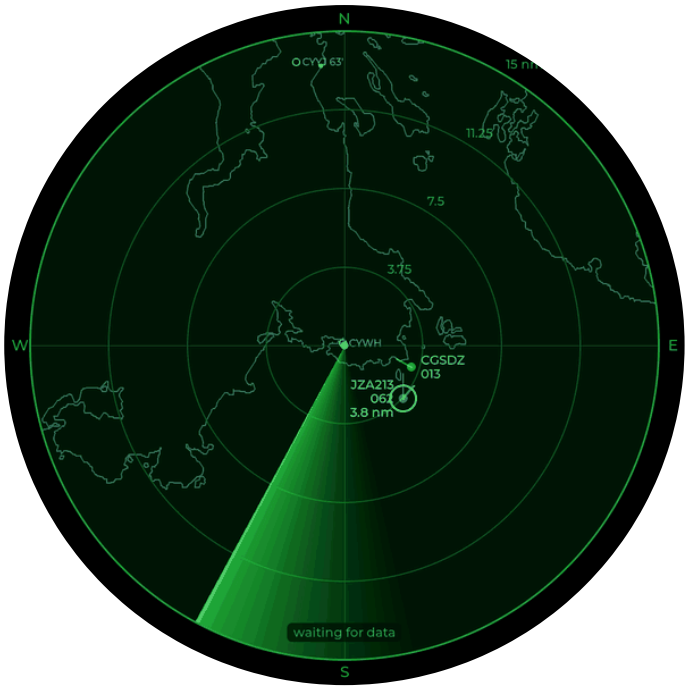
- Each source has its own colour (green for mine, blue for adsb.fi)
- **Red**: squawk 7500 (hijack), 7600 (radio failure) or 7700 (emergency)
- **Grey**: on the ground
- The label shows the callsign and the altitude in hundreds of feet (`062` is 6,200 ft), as on an air traffic control screen

---

Memory

The canvas and its copies are 2.3 MB each (1080 × 1080 × 2 bytes), the bearing table the same again: about 10 MB in all, fine on a 512 MB Zero 2.

6. The map



A plain radar scope is hard to read: is that aircraft over the harbour or over the hills? A faint outline map underneath answers that at a glance. It's made for your own position by `make_map.py`, a Python script that uses only the standard library.

What's in it

Layer	Source	Licence
Shorelines and lakes	GSHHG, full resolution	LGPL
Rivers	GSHHG	LGPL
Airports, runways, frequencies	OurAirports	Public domain
VORs and NDBs (navaids)	OurAirports	Public domain
Towns and peaks (optional)	GeoNames	CC BY 4.0

Navaids (navigation aids) are the radio beacons pilots steer by: VORs, which tell an aircraft its bearing from the station, and older, simpler NDBs. They sit at airports and along airways, so they make good landmarks.

> **Lesson learned:** I tried Natural Earth's coastlines first. They're lovely for a world map, but at 10 nm the islands around Victoria were blobs. GSHHG's full-resolution shorelines show every cove and islet.

Making it

```
./make_map.py --download                # first time: fetch the data too
./make_map.py --airports all            # every airstrip and seaplane base
./make_map.py --towns cities1000.txt --peaks CA.txt US.txt
```

It reads your position from `radar.conf`, keeps everything within 50 nm (`--radius`), simplifies the shorelines to about 37 m, and writes `map.txt`. If a download fails it keeps your old map rather than writing an empty one.

Useful options:

Option	Meaning
`--airports major|coded|all|none`	`coded` (the default) is airports with an ICAO code like CYYJ
`--heliports`	Include heliports
`--no-navaids`, `--no-rivers`, `--no-lakes`	Leave layers out
`--towns FILE --min-population N`	Towns from a GeoNames `citiesNNNN.txt`
`--peaks FILE... --max-peaks N`	The N highest peaks from GeoNames country files

Someone in the Prairies might want towns and every airstrip; someone in the Rockies, the peaks. On the coast, the shoreline says most of it.

The file format

`map.txt` is plain text, so it's easy to inspect or make by other means:

```
> coast                          (a line follows: one lat lon per row)
48.36708 -124.61508
48.36703 -124.61005
...
@ airport 48.64720 -123.42779 CYYJ 63'|Victoria International Airport|...
@ vor 48.72690 -123.48400 YYJ|Victoria|VOR-DME  113.70 MHz, DME channel 84X|...
```

Lines starting `>` begin a shoreline (`coast`), `lake` or `river`; `@` lines are points. After the position comes the label shown on the map, then, separated by `|`, the lines of detail shown when you tap it.

Drawing it

The map is drawn onto the canvas once, under the rings, whenever the range changes. Points are listed most important first, and a label that would overlap one already placed is dropped, so busy areas stay readable at every range. Each point is remembered, so a tap near one can open its details (chapter 7).

7. Touch and screens

The gestures

On the radar	Does
Tap an aircraft	Its details (swipe between them if several were under your finger)
Long-press an aircraft	Follow it; long-press again to stop
Tap an airport or navaid	Its information, METAR and live arrivals board
Tap empty scope	Next range: 5, 10, 15, 20, 25 nm, then back to 5
Swipe left	Clock and weather
Swipe right	Your receiver's stats and range plot
Swipe down	List of aircraft in range
Swipe up	Settings

Every other screen goes back to the radar with a tap or a swipe, or by itself after a while untouched.

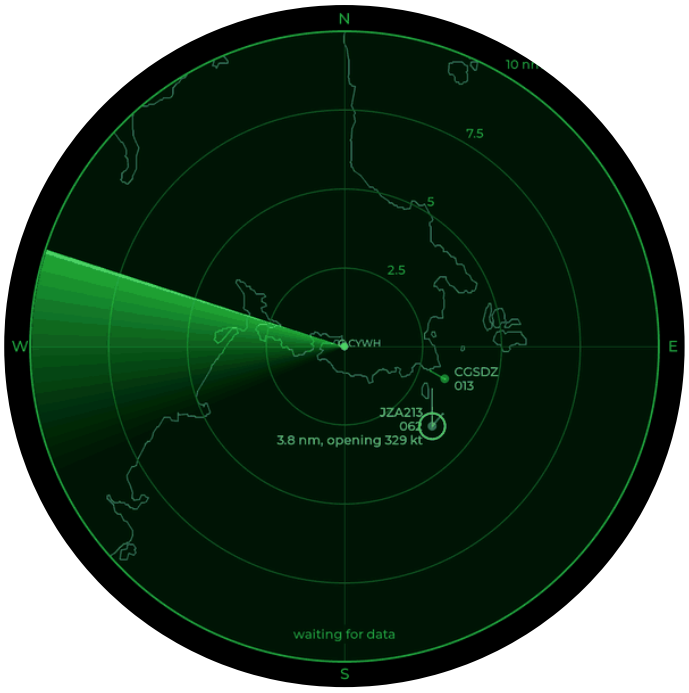
Aircraft details



A tap within 45 pixels of an aircraft opens its details: callsign, altitude, vertical speed, the altitude set on the autopilot, ground speed (its speed over the ground, not through the air), track, squawk, category (light, large, heavy, rotorcraft…), its ICAO address, distance and bearing from you, and signal strength if your receiver heard it. The glossary explains each of these. Type, registration, owner and route are looked up from adsbdb.com in the background ("looking up…") and filled in when they arrive.

If two aircraft are close together, both are opened as pages side by side: swipe between them. The screen goes back to the radar after 20 seconds, or when you tap it.

Follow an aircraft



A long-press puts a white ring round an aircraft and a line under it with its distance from you and whether it's *closing* (getting nearer) or *opening* (moving away), and how fast. It's good for watching an arrival come in.

Airports and navaids



Map points need a closer tap (30 pixels) than aircraft. An airport shows its name, elevation, runways and radio frequencies from ourairports.com, its distance and bearing from you, the latest METAR, and a **live board**: aircraft on the radar whose route, from adsbdb, starts or ends there, with how far out they are.

A navaid shows its identifier, name and type (VOR, VOR-DME, VORTAC, TACAN, NDB or NDB-DME), its frequency, the DME channel if it has one, its elevation, the local magnetic variation, and the airport it serves, with its distance and bearing from you. If your map has towns or peaks, they can be tapped too.

> I wanted real schedules and NOTAMs here, but the sources for those are paid or need agreements. The live board is built from what's already on the scope.

Clock and weather



When the radar has been empty for three minutes it shows the clock and weather instead, and switches back as soon as an aircraft appears. A swipe left shows it any time.

The big clock shows Zulu (UTC), with local time and the zone underneath; either can be the big one. Below are the current conditions with an icon, wind, humidity and pressure, today's high and low, two more days of forecast, and the METAR for your nearest airport, coloured by its flight category: green for

VFR, blue for MVFR, red for IFR and magenta for LIFR (roughly: good, marginal, poor and very poor conditions for flying by sight; the glossary has the details).

Why Zulu? Aviation runs on one clock for the whole world, so a flight plan, a METAR or a controller's instruction means the same time to everyone, with no time zones or daylight saving to convert. That clock is Coordinated Universal Time (UTC), the time at the zero meridian through Greenwich. Time zones have letters, and the zero meridian's is Z, which is "Zulu" in the phonetic alphabet. A METAR issued at 16:00 UTC says `1600Z`.

Your receiver



Swipe right for your receiver's health, from dump1090's `stats.json`: messages per second, signal and noise levels, gain, and the share of signals strong enough to overload the radio. The plot shows how far away your receiver has heard aircraft in each 10° direction: today and the best ever. Hills and buildings show up as dents, so it's a good way to judge an antenna move.

The plot is saved to `range.txt` every five minutes, so the best ever survives restarts and reboots (a power cut can lose the last few minutes). Today's starts afresh at local midnight. The message and signal figures are live from dump1090 and aren't stored.

Aircraft list



Swipe down for every aircraft within the current range, nearest first, in its source's colour. Speeds are ground speeds. Tap a row for its details.

Settings



Swipe up for sliders that change the radar live: the sweep's speed (2–12 seconds a turn), the trail width (0–90°), the frame rate (5–30 per second) and

the track length (0–5 minutes). Below them, how long each sweep frame takes, then three buttons:

- **Update map** runs `make_map.py` in the background and redraws the face
- **Clock & units** (chapter 10)
- **Credits**: everyone whose data and code made this possible

At the bottom, the Pi's vital signs: temperature, CPU (all of it, and the radar's share), memory, disk, power warnings, uptime, IP address (IPv4, or IPv6 on a network without IPv4), and Wi-Fi network and signal strength.

Alerts



When an emergency squawk, a military aircraft or one on your watch list comes within range, its details open by themselves with a red banner (chapter 10 has the settings).

How screens are built

Each screen is an LVGL *screen object*, shown with `lv_screen_load()`. Most are built once at start-up and their labels updated in place; the details and map-point screens are built fresh for each tap (the number of pages varies) and deleted when they close. Swipes are LVGL *gestures*:

```
lv_obj_add_event_cb(radar_scr, radar_event, LV_EVENT_GESTURE, NULL);
...
lv_dir_t dir = lv_indev_get_gesture_dir(lv_indev_active());
if (dir == LV_DIR_LEFT)
    show(S_WEATHER);
```

Two touch bugs worth knowing

The finger that's still down. A swipe switches screens while the finger is still on the glass. The new screen then saw a pressed finger and acted on it: the stats screen closed itself, and settings sliders jumped. The fix is one line after a gesture changes screens, telling LVGL to ignore that finger until it lifts:

```
lv_indev_wait_release(lv_indev_active());
```

The click after a swipe. LVGL sends `CLICKED` when a finger lifts, even at the end of a swipe or a long-press, so a swipe that ended over empty scope also changed the range. The radar now remembers that the current touch was a gesture and ignores the click that follows.

One finger at a time

The screen tracks ten fingers, but LVGL's evdev driver reads only the first, so pinch-to-zoom and two-finger swipes aren't possible without writing a multi-touch driver. Everything here is designed for one finger.

8. Fonts and icons

Built-in fonts

LVGL fonts are normally **bitmaps compiled into the program**: every character pre-drawn at one size. `lv_conf.h` picks which sizes of the Montserrat font to include:

```
#define LV_FONT_MONTSERRAT_16 1
#define LV_FONT_MONTSERRAT_24 1
#define LV_FONT_MONTSERRAT_48 1
```

They're fast and need no files, but each size costs memory, they stop at 48 pixels, and Montserrat is the only face. For a clock you can read from across the room, that's not enough.

TrueType at runtime: Tiny TTF

LVGL includes **Tiny TTF**, which loads an ordinary `.ttf` file and draws its characters at any size as they're needed, keeping recent ones in a cache:

```
// in lv_conf.h:  #define LV_USE_TINY_TTF 1
lv_font_t *big = lv_tiny_ttf_create_data(ttf_bytes, ttf_size, 240);
lv_obj_set_style_text_font(clock_label, big, 0);
```

The radar uses three TrueType fonts, all under the SIL Open Font Licence and all in `radar/fonts`:

Font	Used for
DSEG7 Classic Bold	The big clock: a seven-segment display look
Montserrat Medium	The temperature
Weather Icons	The weather symbols

Any `.ttf` can replace the first two: set `font_file` and `temp_font_file` in `radar.conf`.

Fonts that size themselves

Fonts vary a lot in width. At the same size, DSEG7's digits are far wider than Montserrat's, and the first try ran the clock off the edge of the round screen. So the radar measures a sample ("00:00") and shrinks the font until it fits a box that clears the curved edge (simplified here):

```
lv_font_t *f = lv_tiny_ttf_create_data(data, size, 280);
lv_text_get_size(&s, "00:00", f, 0, 0, LV_COORD_MAX, LV_TEXT_FLAG_NONE);
if (s.x > max_w)                          // too wide: scale it down
    f = lv_tiny_ttf_create_data(data, size, 280 * max_w / s.x);
```

Swap in any font and it still fits.

> **Lesson learned:** I wanted "01:58Z" for Zulu time, but seven segments can't draw a Z: DSEG7's Z is the same shape as a 2. The Z went, and the time zone is on the line below instead.

Weather icons are a font too

Erik Flowers' Weather Icons come as a font: each icon is a character in Unicode's *private use area*, code points set aside for exactly this. Sunny is U+F00D, a night-time shower U+F029, and so on. To show one, put that character in a label set in the icon font. In C that means writing the character as UTF-8, three bytes for these code points:

```
// U+F00D "day-sunny" as UTF-8
lv_label_set_text(icon, "\xEF\x80\x8D");
```

Open-Meteo reports the weather as a **WMO weather code** (0 clear, 3 overcast, 61 light rain, 95 thunderstorm…), and a table maps each to a day or night icon. Because icons are font characters, they take the label's colour, so they're drawn in the same green as everything else, at any size.

A misleading error

After adding the icons, the log filled with:

```
ttf_get_glyph_dsc_cb: cache not allocated
```

It wasn't about a cache. A new LVGL label starts with the placeholder text "Text", and the code set the icon font *before* clearing that text. LVGL then measured "T", "e", "x", "t" in a font that has only weather symbols, and Tiny TTF reports a missing character with that message. Clearing the text before setting the font fixed it.

9. Where the data comes from

Everything comes over plain HTTP(S) as JSON, fetched with **libcurl** and read with **cJSON**, a small C JSON parser. None of these services needs an account or a key.

Source	Gives	Used for	Terms
Your receiver's `aircraft.json`	Every aircraft it hears, each second	The radar	Yours
adsb.fi open data	Aircraft near a point, from thousands of feeders	Filling in what your antenna misses; registration and type	Personal, non-commercial use; at most 1 request a second; credit adsb.fi
adsbdb.com	An aircraft's type, registration and owner; a flight's route	Details screen, airport board	Free; keep requests light
Open-Meteo	Current weather and forecast for a point	Clock & weather screen	Free for non-commercial use; CC BY 4.0, credit Open-Meteo
aviationweather.gov (NOAA)	METARs, worldwide	Weather and airport screens	US government, public
Your receiver's `stats.json`	Message rates and signal levels	Receiver screen	Yours

Your receiver

dump1090 writes `aircraft.json` once a second. One aircraft looks like this (trimmed):

```
{ "hex": "c0184b", "flight": "JZA213  ", "alt_baro": 6200, "gs": 212.4,
  "track": 40.1, "lat": 48.4000, "lon": -123.3000, "squawk": "4521",
  "baro_rate": -832, "category": "A2", "rssi": -18.6, "seen_pos": 0.4 }
```

`seen_pos` is how many seconds ago the position was heard; anything over a minute old is ignored so stale aircraft don't linger.

adsb.fi

adsb.fi gathers data from volunteers' receivers worldwide and offers it in the same format. Its URL takes your position and a radius:

```
url = https://opendata.adsb.fi/api/v3/lat/{lat}/lon/{lon}/dist/{dist}
```

The radar fills in `{lat}` and `{lon}` from `radar.conf`, and `{dist}` as the widest range plus 20%, so aircraft don't pop in and out at the edge. It polls every 3 seconds, well within the one-a-second limit, and the status line names adsb.fi as the terms ask.

Its records add fields your receiver can't know, such as registration (`r`), type (`t`), a description, owner, and `dbFlags`, whose lowest bit marks military aircraft. The alerts use that.

adsbdb

When you tap an aircraft, a background thread asks adsbdb about it:

```
https://api.adsbdb.com/v0/aircraft/C0184B      type, registration, owner
https://api.adsbdb.com/v0/callsign/JZA213      airline, origin, destination
```

Answers are cached, so each aircraft and flight is asked about only once. Routes are only looked up for airline-style callsigns (three letters then a number, like JZA213), not for registrations used as callsigns.

Open-Meteo

One request every 15 minutes brings the current temperature, humidity, WMO weather code, wind (in knots), pressure and whether it's day or night, plus highs, lows, weather codes and chance of rain for the next few days.

METARs

```
CYYJ 031600Z 25012G22KT 15SM -RA FEW018 BKN040 OVC080 12/09 A2990
```

Victoria, on the 3rd at 16:00 UTC: wind from 250° at 12 knots gusting 22, 15 statute miles visibility, light rain, cloud layers at 1,800, 4,000 and 8,000 feet, temperature 12 °C, dew point 9 °C, altimeter 29.90 inches. The JSON from aviationweather.gov also gives the flight category (VFR, MVFR, IFR, LIFR), which colours it on screen.

What I left out

- **Flight schedules** for the airport boards: the good sources are paid. The live board uses adsbdb routes for aircraft already on the scope.
- **NOTAMs**: Nav Canada has no open API, and the FAA's needs registration.
- **Aircraft photos**: each photo belongs to its photographer, and the terms for showing them in an app weren't clear, so I skipped them.

> **A good habit:** every one of these is a free service run by people who asked only for a credit and light use. The radar polls no faster than it needs, caches lookups, backs off when a source fails, and lists them all on the Credits screen. Aggregators like adsb.fi exist because people feed them, so I'm planning to give back by feeding my own receiver's data to adsb.fi and airplanes.live.

IPv4 and IPv6

The radar fetches with libcurl and the Python tools with Python's own library. Both handle IPv6 and IPv4 alike: given a name with both kinds of address, they try IPv6 first and fall back to IPv4. On a home network with both (*dual-stack*), everything works whichever the services offer. On a network with IPv6 only, the IPv4-only services below are reachable only if the network provides NAT64/DNS64, a translation service that most IPv6-only networks, mobile carriers included, do offer.

Has IPv6	IPv4 only
adsbdb.com (lookups)	adsb.fi (aircraft)
aviationweather.gov (METARs)	Open-Meteo (weather)
raw.githubusercontent.com (GSHHG shoreline data)	GeoNames (towns and peaks)
OurAirports data (on GitHub Pages)	github.com (cloning the repo)

This was checked in October 2026; services add IPv6 over time. On your own network the receiver, SSH and the screen mirror (chapter 11) all work over IPv6 too.

10. Settings

radar.conf

All settings live in `radar.conf`, a plain text file of `key = value` lines, commented throughout. Comments can go at the end of a line too:

```
unit_temp = C          # C or F
```

When you change something on screen (the range, a slider, a unit), the radar writes it back into `radar.conf` in place, keeping your comments and layout, so the change survives a restart.

The main groups:

Settings	What they do
`lat`, `lon`	Your receiver's position
`range_nm`, `range_min`, `range_max`, `range_step`	The starting range and the steps a tap cycles through
`sweep_period_s`, `sweep_trail_deg`, `sweep_fps`	Sweep speed, trail width and frame rate
`trail_s`	Seconds of track drawn behind each aircraft
`map_file`, `map_options`, `show_*`, `map_*_colour`	The map, what's shown on it, and its colours
`detail_timeout_s`, `idle_after_s`, `weather_view_s`	When screens time out
`font_file`, `temp_font_file`, `icon_font_file`	Fonts (chapter 8)
`clock_main`, `clock_24h`, `unit_*`	Clock and units
`metar_station`	The airport whose METAR is on the weather screen
`alert_emergency`, `alert_military`, `watch`	Alerts
`[source]` sections	Where aircraft come from, how often, and in what colour
`fb_dev`, `touch_dev`	The screen and touch devices

The web addresses for weather, METARs, lookups and receiver stats can be changed too (`weather_url`, `metar_url`, `lookup_url`, `stats_url`).

Clock and units



Settings › **Clock & units** has a row of buttons for each:

	Choices (default first)
Big clock	Zulu (UTC), local
Hours	24 h, 12 h
Temperature	°C, °F
Speed	knots, km/h, mph
Altitude	feet, metres
Distance	nautical miles, km, statute miles
Pressure	hPa, kPa, inHg

The defaults are aviation's (and Canada's): Zulu (UTC), 24-hour, Celsius, knots, feet, nautical miles and hectopascals. Altitudes from 18,000 ft up are shown as flight levels (FL350) as pilots say them. The range rings always stay in nautical miles.

Alerts

Three kinds of aircraft can open their details by themselves when they come within range:

- **Emergencies**: squawk 7500, 7600 or 7700 (`alert_emergency = yes`)
- **Military** aircraft, as flagged by adsb.fi (`alert_military = yes`)
- **Your watch list**:

```
watch = C-FTCA, RCH*, C130, A7615F
```

Entries can be a registration, a callsign, an ICAO type or an ICAO address, in any case, with or without dashes. A `*` at the end matches the start, so `RCH*` catches every US Air Force "Reach" flight. Each aircraft alerts at most once every half hour.

11. Running it for real

Why it "crashed" after a while

For the first few days I started the radar from an SSH session. Every so often the screen dropped back to the text console, and my SSH window said "broken pipe". The radar hadn't crashed: when the SSH connection dropped, Linux sent the programs started from it a **hangup signal** (SIGHUP), which ends a program by default. The radar's signal handler politely gave the screen back to the console on the way out.

The fix is not to run it from a login session at all, but as a **systemd service**.

The service

```
cd lvgl-radar/radar
./install-service.sh
```

That writes `/etc/systemd/system/radar.service` for your user and folder, enables it to start at boot, and starts it now. It also *masks* the login prompt on the screen (`getty@tty1`), so it can't draw over the radar; `./install-service.sh remove` puts it back. The interesting lines:

```
[Unit]
Wants=network-online.target
After=network-online.target
Conflicts=getty@tty1.service        # no login prompt on the radar's screen

[Service]
User=gordo                          # not root
WorkingDirectory=/home/gordo/lvgl-radar/radar
ExecStart=/home/gordo/lvgl-radar/radar/build/radar radar.conf
Restart=always                      # if it ever stops, start it again
SupplementaryGroups=video input tty
AmbientCapabilities=CAP_SYS_TTY_CONFIG
```

> **Lesson learned:** after the first reboot with the service, boot text showed around the edges of the radar. Moving the boot messages to another console (chapter 2) fixed it. The installer also masks the login prompt, since `Conflicts=` alone can't stop it starting at the same time as the radar.

The groups give access to the framebuffer, the touch screen and the console. The capability allows exactly one privileged thing: switching the console to graphics mode (chapter 1). So the radar runs as an ordinary user, not as root.

Day to day:

```
sudo systemctl restart radar       # after rebuilding or editing radar.conf
sudo systemctl stop radar          # e.g. to run the touch demo
journalctl -u radar -f             # its messages, live
./install-service.sh remove        # take it out again
```

Wi-Fi that stays awake

The Pi's Wi-Fi saves power by dozing between packets, which can make SSH sluggish and drop connections. With NetworkManager (the Pi OS default), turn it off for your network:

```
nmcli connection show                                  # your network's name
sudo nmcli connection modify "NAME" 802-11-wireless.powersave 2
```

(`2` means "disable".) It takes effect the next time the connection comes up.

Taking it out

Away from home your own receiver can't be reached, but with a phone hotspot the Pi still gets adsb.fi's live traffic around the position in `radar.conf`, plus the weather. Join the hotspot once at home so the Pi knows it:

```
sudo nmcli dev wifi connect "HOTSPOT NAME" password "PASSWORD"
```

The network also sets the clock: the Pi has no battery-backed clock, and without a network it starts from roughly when it was last shut down.

To show the sky over wherever you are instead, change `lat` and `lon` in `radar.conf`, run `./make_map.py` for the new spot, and restart the radar.

Showing the screen to a room

A 5-inch screen is hard to see from the back row. Two ways to put it on a projector or a video call:

An HDMI splitter sends the Pi's picture to both the round screen and a projector. It only works if the projector accepts a 1080 × 1080 picture, so test it before the day.

fbmirror, in the repo's `tools/` folder, shows the framebuffer live in a web browser:

```
sudo apt install python3-pil
./tools/fbmirror.py
```

Then open `http://radar.local:8081/` on a laptop on the same network (the Pi's hostname, plus `.local`). It listens on IPv6 as well as IPv4, so `http://[the Pi's IPv6 address]:8081/` works too. It reads `/dev/fb0` four times a second, converts the RGB565 pixels with Pillow, and sends each changed frame as a JPEG in a *Motion JPEG* stream, which browsers show as live video with no plugins. Share that browser tab in Jitsi or Zoom and remote viewers see the screen too.

It works for anything on the framebuffer: the touch demo, the radar, even the text console.

12. Credits, lessons and ideas

Credits



Data

- adsb.fi: open aircraft data from volunteer feeders, for personal, non-commercial use
- adsbdb.com: aircraft and route lookups
- Open-Meteo: weather, CC BY 4.0
- aviationweather.gov (NOAA): METARs
- GSHHG: shorelines and rivers, by P. Wessel and W. H. F. Smith, LGPL
- OurAirports: airports, runways, frequencies and navaids, public domain
- GeoNames: towns and peaks, CC BY 4.0

Software and fonts

- LVGL: MIT licence
- libcurl: curl licence
- cJSON: MIT licence
- DSEG fonts by keshikan: SIL OFL 1.1
- Weather Icons by Erik Flowers and Lukas Bischoff: SIL OFL 1.1
- Montserrat, by the Montserrat Project Authors: SIL OFL 1.1

The code in this repository is BSD 2-Clause licensed.

Lessons learned

- **Check the cable.** A blank screen was a half-seated mini-HDMI plug.
- **`video=` in cmdline.txt**, not `hdmi_*` in config.txt, sets the mode with today's graphics driver.
- **Rebuild from clean when things make no sense.** Once, after big changes, aircraft refused to blink and tracks wouldn't draw. An old build was being run. `rm -rf build` and building again fixed it.
- **Programs started over SSH die with the session.** Use a service.
- **Keep LVGL on one thread.** Worker threads fetch; the main thread draws.
- **Gestures need care**: wait for the finger to lift after switching screens, and ignore the click that ends a swipe.
- **Read error messages sceptically.** "cache not allocated" meant "this font has no letter T".
- **Old guides and new versions.** LVGL 9 renamed its CMake options, and the old names were silently ignored, so LVGL's examples and demos were being built too. Check the version a guide was written for.
- **Data comes with terms.** Every free service here asks for a credit and light use. It's easy to honour, and it keeps them free.

Ideas for next time

- **Weather alerts**: Environment Canada publishes alerts as open data; a coloured ring around the scope could show a warning in effect.
- **Pinch to zoom**: needs a multi-touch input driver, since LVGL's evdev driver reads one finger.
- **A bigger Pi**: on a Pi 4 or 5, a higher frame rate and smoother sweep.
- **Other screens**: anything with a framebuffer works, from little SPI panels on `/dev/fb1` to large HDMI monitors. Only the layout assumes a 1080-pixel circle.
- **Other data**: AIS for ships, APRS for radio amateurs, or lightning strikes, all could be plotted on the same scope.

Thanks

To the Raspberry Pi meetup for listening, to everyone who feeds adsb.fi, and to the people behind every library, font and data set listed above.

13. Glossary

Aviation and weather

- **ADS-B** (Automatic Dependent Surveillance–Broadcast): aircraft broadcast their own identity, position, altitude and speed about twice a second on 1090 MHz. Anyone with a receiver can listen.
- **Airspeed**: speed through the air. Not shown here; see *ground speed*.
- **Altitude**: in the details, the *pressure altitude* the transponder reports, measured as if the air pressure at sea level were the standard 29.92 inHg (1013.25 hPa). Close to height above sea level, but it varies with the weather.
- **ATIS, TWR, GND, CLD, APP, ARR, DEP, MF, RCO, CTAF**: the radio frequencies on an airport's screen. ATIS (Automatic Terminal Information Service) is a recorded weather and runway broadcast; TWR is the tower; GND is ground control (taxiing); CLD is clearance delivery (flight plan approvals); APP, ARR and DEP are approach, arrival and departure control; MF is a Canadian *mandatory frequency* at airports without a tower; RCO is a *remote communications outlet* for reaching a flight service station; CTAF is the *common traffic advisory frequency* pilots use to announce themselves at uncontrolled fields.
- **Bearing**: the direction from you to something, in degrees clockwise from true north (090° is east).
- **Callsign**: the name an aircraft uses on the radio and in ADS-B. Airlines use their ICAO code and a flight number: JZA213 is Jazz flight 213. Private aircraft usually use their registration, like CGSDZ.
- **Category**: the size or kind of aircraft, as broadcast: A1 light (under 15,500 lb), A2 small, A3 large, A4 a large aircraft with strong wake turbulence (the Boeing 757), A5 heavy (over 300,000 lb), A6 high performance, A7 rotorcraft, and B and C codes for gliders, balloons, drones and airport vehicles.
- **Ceiling**: the height of the lowest cloud layer covering more than half the sky (*broken* or *overcast*).
- **Closing and opening**: whether an aircraft is getting nearer to you (closing) or moving away (opening), and how fast its distance is changing, in knots.
- **DME** (Distance Measuring Equipment): a beacon, often paired with a VOR, that tells an aircraft its distance from the station.
- **Flight category**: how good the weather is for flying by sight, from a METAR's ceiling and visibility: **VFR** (Visual Flight Rules): ceiling above 3,000 ft and

visibility over 5 miles. **MVFR** (Marginal VFR): ceiling 1,000–3,000 ft or visibility 3–5 miles. **IFR** (Instrument Flight Rules): ceiling 500 to under 1,000 ft or visibility 1 to under 3 miles. **LIFR** (Low IFR): ceiling under 500 ft or visibility under 1 mile. Visibility is in statute miles.

- **Flight level**: altitude in hundreds of feet at standard pressure, used high up so everyone's altimeters agree: FL350 is 35,000 ft. In Canada's southern airspace and in the US, flight levels start at 18,000 ft.
- **Ground speed**: speed over the ground, in knots. It's airspeed plus or minus the wind, which is why a westbound airliner into a headwind can be 100 knots slower than an eastbound one.
- **hPa, kPa, inHg**: units of air pressure: hectopascals (the same as millibars), kilopascals, and inches of mercury (used in North American altimeter settings). 1013 hPa = 101.3 kPa = 29.92 inHg.
- **ICAO** (International Civil Aviation Organization): the United Nations agency that sets world aviation standards.
- **ICAO address** (shown as *Hex*): a 24-bit number unique to each aircraft's transponder, written as six hexadecimal digits like C0184B. Blocks of addresses belong to countries; C0 to C3 are Canada's.
- **ICAO airport code**: four letters, like CYYJ for Victoria International. The first letter is the region: C for Canada, K for the contiguous US. The three-letter codes on luggage tags (YYJ) are IATA codes.
- **ICAO type designator**: a short code for an aircraft model: DH8D is the Dash 8-400, B738 the Boeing 737-800, C172 the Cessna 172.
- **Knot**: one nautical mile per hour, 1.852 km/h.
- **Magnetic variation**: the angle between true north and magnetic north. Around Victoria, magnetic north is about 15–18° east of true north.
- **METAR** (Meteorological Aerodrome Report): the standard routine airport weather report, usually issued every hour. Chapter 9 decodes one.
- **Military flag**: adsb.fi marks aircraft its database lists as military, which the radar can alert on.
- **Nautical mile**: 1,852 m, or one minute of latitude. The radar's range rings are in nautical miles.
- **Navaid** (navigation aid): a radio beacon pilots navigate by, such as a VOR or NDB.
- **NDB** (Non-Directional Beacon): a simple low-frequency beacon; a direction finder in the aircraft points at it. An **NDB-DME** adds distance.
- **NOTAM**: a notice to pilots of a temporary hazard or change, such as a closed runway.

- **Registration**: an aircraft's licence plate, painted on its side: C-GKQJ (C- for Canada, N for the US). Also called the tail number.
- **Selected altitude**: the altitude the pilots have set on the autopilot, which the aircraft is climbing or descending to.
- **Squawk**: the four-digit code the pilot sets on the transponder, given by air traffic control. 1200 means flying by sight (VFR) in North America; 7500 means hijack, 7600 radio failure and 7700 emergency.
- **TACAN** (Tactical Air Navigation): the military equivalent of a VOR and DME. A **VORTAC** is a VOR and TACAN at one site.
- **Track**: the direction an aircraft is moving over the ground, in degrees. It can differ from its *heading* (where the nose points) because of the wind.
- **Transponder**: the radio in an aircraft that answers air traffic control radar and, with ADS-B, broadcasts its position.
- **UTC** (Coordinated Universal Time): the world's reference time, the time at the zero meridian through Greenwich, with no daylight saving.
- **Vertical speed**: how fast an aircraft is climbing (+) or descending (−), in feet per minute.
- **VOR** (VHF Omnidirectional Range): a beacon that tells an aircraft its bearing from the station. A **VOR-DME** adds distance.
- **WMO weather code**: a number from the World Meteorological Organization describing the weather (0 clear, 3 overcast, 61 light rain, 95 thunderstorm). Open-Meteo reports one, and it picks the weather icon.
- **Zulu**: aviation's name for UTC, from the time zone letter Z, "Zulu" in the phonetic alphabet. `1600Z` is 16:00 UTC.

Computing

- **Aggregator and feeder**: an aggregator such as adsb.fi combines the data from many people's receivers, the feeders, into one picture.
- **dBFS** (decibels relative to full scale): signal strength at the receiver, where 0 is the loudest it can measure and lower (more negative) numbers are weaker. A *strong* signal is one near 0 that can overload the radio.
- **Dual-stack**: a network that has both IPv4 and IPv6.
- **dump1090**: the program that decodes ADS-B from a cheap USB radio and publishes `aircraft.json`. dump1090-fa is FlightAware's version.
- **evdev**: Linux's interface for input devices, `/dev/input/event*`.
- **Framebuffer**: the screen's memory as a file, `/dev/fb0`.
- **Gesture**: a swipe, which LVGL reports as up, down, left or right.

- **KMS** (Kernel Mode Setting): how Linux sets the display's resolution; the `video=` setting in `cmdline.txt` goes to it.
- **LVGL** (Light and Versatile Graphics Library): the C library that draws everything on the screen.
- **mDNS**: how `radar.local` finds the Pi on the local network without a DNS server. Pi OS's avahi-daemon answers it, for IPv4 and IPv6.
- **Motion JPEG**: video sent as a stream of JPEG pictures, which browsers show without plugins. The screen mirror uses it.
- **Mutex**: a lock that lets one thread at a time use shared data.
- **NAT64 and DNS64**: a translation service that lets devices on an IPv6-only network reach IPv4-only servers.
- **Phosphor**: the glowing coating on an old radar or TV tube, which faded slowly; the radar's fading blips imitate it.
- **RGB565**: a pixel in 16 bits: 5 for red, 6 for green, 5 for blue.
- **Submodule**: a git repository included in another at a fixed version; LVGL is one in this repository.
- **systemd service**: a program Linux starts at boot and restarts if it stops.
- **Thread**: a part of a program that runs at the same time as the rest.
- **Tiny TTF**: LVGL's way of drawing TrueType fonts at any size.